

Bruchrechnen mit Python

Ein Hackathon

Polu Tartakower

Kiel, September 2025

Inhaltsverzeichnis

1	Übersicht	5
1.1	Die Aufgabenstellung	5
1.2	Notwendige Programmiererfahrung	5
1.3	Mathematische Vorkenntnisse	6
1.4	Eingesetzte Technologien	6
1.5	Methoden der Softwareentwicklung	6
2	Die Entwicklungsumgebung	8
2.1	Installationsquellen und Lizenzbedingungen .	8
2.2	Das SDK für Python	9
2.3	Die integrierte Entwicklungsumgebung (IDE) .	9
2.3.1	VS Code	10
2.4	Die Projektstruktur	10
3	Die integrierte Entwicklungsumgebung VS Code	11
3.1	Installation von VS Code	11
3.2	Deutsche Sprachunterstützung	11
3.3	Ein Profil für Python-Programmierung anlegen	11
3.4	Sprachunterstützungen für Python	12
3.5	Die Befehlspalette	12
3.6	Einstellungen von VS Code und den Plugins .	12
4	Einrichten des Entwicklungsprojektes	13
4.1	Der Hauptordner des Projektes	13
4.2	Der Arbeitsbereich in VS Code	13
4.3	Die Ordner für den Quellcode	13
4.4	Die Pfadangabe für die Python-Module	13
4.5	Die virtuelle Entwicklungsumgebung	14

4.6	Konfiguration der Softwaretest und Installation des Testframeworks	14
4.7	Installation des GUI-Frameworks NiceGui ...	15
5	Die Softwarearchitektur	16
5.1	Die Problemstellung	16
5.2	Die Lösung	16
5.3	Trennung von funktionalem Quellcode und Testcode	16
5.4	Trennung von Logik und Darstellung	16
6	Eine Funktion in Python: Programmierung und Testen	18
6.1	Die Funktion zur Berechnung des größten gemeinsamen Teilers	18
6.2	Der Unittest	18
7	Lösung: Programmieren und Testen des ggT	20
7.1	Die Funktion zur Berechnung des ggT	20
7.2	Sinnvolle Testwerte	20
7.3	Die Testfunktion	20
8	Bruch: Objektorientierte Analyse und objektorientiertes Design (OOAD)	21
8.1	Aufgabenstellung OOAD	21
9	Lösung: Objektorientierte Analyse und objektorientiertes Design (OOAD)	22
9.1	Eigenschaften eines Bruchs	22
9.2	Vorzeichen in Zähler und Nenner	22
9.3	Der Bruchstrich	22

9.4	Die Gleichheit von Brüchen	22
9.5	Die Unveränderlichkeit von Brüchen	23
9.6	Das Kürzen von Brüchen	24
9.7	Das Erweitern von Brüchen	24
9.8	Das Multiplizieren von Brüchen	24
9.9	Das Dividieren von Brüchen	24
9.10	Das Addieren von Brüchen	25
9.11	Das Subtrahieren von Brüchen	25
10	Die Klasse Bruch	26
10.1	Objektorientierung in Python	26
10.2	Die Klassendefinition	26
10.3	Die Deklaration der Attribute	26
10.4	Der Konstruktor zur Objekterzeugung	26
10.5	Eine Funktion zur Berechnung des Zahlenwertes	26
10.6	Eine Funktion für einen positiven Nenner ...	27
11	Lösung: Die Klasse Bruch	28
11.1	Die Klassen und ihre Attribute	28
11.2	Der Konstruktor zur Objekterzeugung	28
11.3	Eine Funktion zur Berechnung des Zahlenwertes	28
11.4	Eine Funktion für einen positiven Nenner ...	29
12	Die Klasse Bruch: Weitere Funktionen	30
13	Die graphische Benutzeroberfläche	31
14	Ausbaustufen	32
14.1	Eine Klasse für Kommazahlen	32
14.2	Eine Klasse für unechte Brüche	32

1 Übersicht

1.1 Die Aufgabenstellung

Es soll eine Webanwendung programmiert werden, mit der beispielsweise folgende Bruchrechenoperation berechnet werden kann:

$$\frac{1}{2} \cdot \frac{2}{3}$$

Die Ergebniszeile soll so aussehen:

$$\frac{1}{2} \cdot \frac{2}{3} = \frac{2}{6} = \frac{1}{3}$$

Das soll auf der Oberfläche so gestaltet werden:

- Auf der Webseite gibt es je ein Eingabefeld für den Nenner und für den Zähler der beiden Operanden.
- Die Rechenoperation kann aus einer Dropdown-Liste ausgewählt werden, welche die vier Grundrechenarten enthält.
- Das erste Gleichheitszeichen ist ein Button, dessen Betätigung die Berechnung ausführt.
- Nach dem ersten Gleichheitszeichen wird das Ergebnis ungekürzt dargestellt. Das gekürzte Ergebnis folgt nach dem zweiten Gleichheitszeichen.

1.2 Notwendige Programmiererfahrung

Du solltest erste Erfahrungen mit der Programmierung besitzen, konkret folgende Sprachkonstrukte bereits angewendet haben:

- Variable und Konstante
- Wertzuweisungen
- Bedingte Anweisungen
- Schleifen

Falls du noch nie programmiert hast, wird der Hackathon vermutlich zu schwierig für dich sein. Dann solltest du dich vorher unbedingt mit den grundlegenden Sprachelementen von Python vertraut machen.

1.3 Mathematische Vorkenntnisse

Außerdem musst du ein grundlegendes Wissen über das Rechnen mit Brüchen besitzen:

- Was ist ein Bruch? Woraus besteht er?
- Welche Bedeutung besitzt der Bruchstrich?
- Was ist ein gekürzter Bruch? Wie wird ein Bruch gekürzt?
- Wie wird ein Bruch erweitert?
- Wie werden die Grundrechenoperationen mit Brüchen durchgeführt?

Wenn du noch nie mit Brüchen gerechnet hast oder dein Wissen nicht mehr frisch ist, muss du unbedingt zuvor das Rechnen mit Brüchen lernen oder wiederholen.

1.4 Eingesetzte Technologien

Für die Programmierung verwenden wir die Programmiersprache Python. Wenn du bisher nur mit einer anderen Sprache programmiert hast, beispielsweise mit Scratch, wirst du dich schnell in Python zurecht finden.

Für die Programmierung der Web-Oberfläche verwenden wir das GUI-Framework NiceGui. Du musst also keine Vorkenntnisse in HTML, CSS oder JavaScript besitzen, sie helfen aber auf jeden Fall.

1.5 Methoden der Softwareentwicklung

Im Rahmen des Hackathons soll nicht nur programmiert werden, sondern auch verschiedene moderne Methoden, wie in der Softwareentwicklung

heutzutage gearbeitet wird, angewendet werden. Dazu sollst du im Team zusammen mit anderen folgende Aufgaben bearbeiten:

- Erstellung einer Entwicklungsumgebung (Abschnitt 2)
- Entwurf einer Softwarearchitektur
- Programmierung einer Funktion zur Berechnung des größten gemeinsamen Teilers (ggT)
- Programmierung eines Unittests für die ggT-Funktion
- Objektorientierte Analyse und Design für einen Bruch
- Implementierung der Klasse Bruch
- Programmierung der Unittests für die Klasse Bruch
- Programmierung der Benutzeroberfläche

2 Die Entwicklungsumgebung

Unsere Entwicklungsumgebung umfasst folgende Komponenten:

- das Software Developer Kit (SDK), also die Programmiersprache Python einschließlich des Interpreters und vieler Bibliotheken
- eine integrierte Entwicklungsumgebung (**IDE**) zum Editieren von Quellcode
- eine einheitliche Ordnerstruktur im Dateisystem für Softwareentwicklungsprojekte
- eine virtuelle Umgebung
- ein Testframework
- weitere Bibliotheken von Drittanbietern

Wer bereits Erfahrung mit den Werkzeugen für die Softwareentwicklung hat, kann folgende Komponenten zusätzlich integrieren:

- eine Software zur Verwaltung des Python-Projektes
- eine lokale Versionsverwaltung
- einen Account für eine cloudgestützte Versionsverwaltung

2.1 Installationsquellen und Lizenzbedingungen

Wir werden verschiedene Tools zur Unterstützung der Programmierung herunterladen und installieren. Dabei sind zwei Aspekte sehr, sehr wichtig:

Verwende nur seriöse Installationsquellen!

Die Installation von Software auf dem Computer birgt immer das Risiko, den Computer mit Schadsoftware zu infizieren. Das muss unbedingt vermieden werden!

Die sichersten Installationsquellen bilden entweder die Paketverwaltung des Betriebssystems oder die Webseite des Herstellers.

Halte die Lizenzbedingungen der Software ein!

Jede Software enthält eine Lizenz, deren Bedingungen bei Nutzung unbedingt eingehalten werden müssen. Andernfalls begeht man als Nutzer eine Straftat! Das gilt auch für freie und kostenlose Software! Informiere dich über die jeweiligen Lizenzen und halten deren Bedingungen unbedingt ein.

2.2 Das SDK für Python

In einer aktuellen Linux-Installation sollte in der Regel eine aktuelle Version von Python bereits installiert sein. Unter MS Windows kann eine aktuelle Version von Python über den Microsoft Shop installiert werden.

Aufgaben:

- Recherchiere, welche Versionen von Python als aktuell gelten.
- Finde heraus, wie auf einem Computer geprüft werden kann, ob Python installiert ist und gegebenenfalls welche Version vorliegt.

2.3 Die integrierte Entwicklungsumgebung (IDE)

Eine integrierte Entwicklungsumgebung (Integrated Developer Environment, **IDE**) enthält als Kernkomponente einen Editor zum Erstellen und Bearbeiten von Quellcode. Dieser Codeeditor unterstützt die Entwicklerin durch farbige Markierung des Quelltextes und laufende Prüfungen beim Programmieren.

Darüber hinaus besteht eine IDE aus weiteren zahlreichen Hilfsprogrammen für die Softwareentwicklung, z.B. zur Bedienung der Versionsverwaltung oder zur Ausführung der Testprogramme. Alle Funktionalitäten sind benutzerfreundlich innerhalb einer Bedienoberfläche zusammen gefasst.

Tipp: Verwende zum Programmieren unbedingt eine für die jeweilige Programmiersprache optimierte IDE.

Für Python existieren zwei sehr gut geeignete IDE:

- Microsoft Visual Code (VS Code)

- JetBrains Pycharm

Bei diesem Hackathon verwenden wir VS Code. Falls du bereits Erfahrung mit Pycharm besitzt, darfst du natürlich weiterhin Pycharm verwenden.

2.3.1 VS Code

Informiere dich über VS Code. Aufgaben:

- Woher kannst du VS Code beziehen?
- Wie viel kostet es?
- VS Code allein ist gar keine IDE, sondern nur ein Code-Editor. Wie wird VS Code zur vollwertigen IDE?

Die Installation und Konfiguration von VS Code behandeln wir im nächsten Kapitel. (Abschnitt 3)

2.4 Die Projektstruktur

Der Aufbau der Projektstruktur wird im übernächsten Kapitel beschrieben. (Abschnitt 4)

3 Die integrierte Entwicklungsumgebung VS Code

Das Tool Microsoft Visual Studio Code (**VS Code**) ist streng genommen nur ein Code-Editor, der allerdings durch die Integration von verschiedenen Plugins zu einer integrierten Entwicklungsumgebung (**IDE**) wird.

3.1 Installation von VS Code

VS Code kann entweder über die Paketverwaltung des Betriebssystems oder die Herstellerseite bezogen werden. Aus der Webseite finden sich auch umfangreiche Dokumentationen, insbesondere zur Installation.

3.2 Deutsche Sprachunterstützung

Wir werden in diesem Text die deutschen Bezeichnungen benutzen. Daher empfehlen wir die Installation des Plugins *German Language Pack* von Microsoft.

Wer bereits größere Erfahrung hat, darf natürlich mit den englischen Bezeichnungen arbeiten.

3.3 Ein Profil für Python-Programmierung anlegen

Wer mit verschiedenen Programmiersprachen in VS Code arbeiten möchte, benötigt jeweils spezielle Plugins für diese Programmiersprachen. Um die unterschiedlichen Plugins voneinander getrennt zu halten, empfiehlt sich die Einrichtung eines Profils für jeweils jede Programmiersprache.

Im Navigator links unten über das Zahnrad das Kontextmenü öffnen, dort *Profil* auswählen.

3.4 Sprachunterstützungen für Python

Falls du ein Profil für die Programmiersprache Python angelegt hast, aktiviere dieses.

Installiere das Plugin *Python* von Microsoft.

Dadurch werden zwei weitere Plugins installiert, die wir benötigen.

- *Pylance*
- *Python Debugger*

Falls zusätzlich das Plugin *Python Environments* installiert wird, diese bitte wieder löschen! Es ist von sehr schlechter Qualität und schadet nur.

3.5 Die Befehlspalette

Wichtige Funktionen von VS Code und der Python-Plugins werden über die Befehlspalette ausgeführt. Diese kann auf zwei Wegen aufgerufen werden:

- Menü *Anzeigen*, erster Eintrag *Befehlspalette...*
- Tastenkombination STRG+UMSCHALT+P

3.6 Einstellungen von VS Code und den Plugins

VS Code und die Plugins lassen sich umfangreich konfigurieren. Das geschieht über die Einstellungen, die auf folgenden Wegen aufgerufen werden können:

- Im Navigator links unten über das Zahnrad das Kontextmenü öffnen.
- Tastenkombination STRG+,

4 Einrichten des Entwicklungsprojektes

In diesem Kapitel werden wir die Infrastruktur für unser Entwicklungsprojekt einrichten. Das sollte überwiegend in VS Code durchgeführt werden, damit alle Komponenten reibungslos zusammen arbeiten.

4.1 Der Hauptordner des Projektes

Lege im Dateisystem unter dem Ordner *Programmierprojekte* einen neuen Ordner *Bruchrechnen* an. Dieser neue Ordner wird der Hauptordner des Projektes sein.

Ab jetzt werden alle Einstellungen in VS Code vorgenommen.

4.2 Der Arbeitsbereich in VS Code

- Starte VS Code.
- Öffne dort den Ordner *Bruchrechnen*.
- Lege einen Arbeitsbereich an: Im Menüpunkt *Datei/Arbeitsbereich speichern unter...* den Vorschlag annehmen.
- Der Arbeitsbereich lässt sich zukünftig über die Datei *Bruchrechnen.code-workspace* öffnen und schließen.

4.3 Die Ordner für den Quellcode

Lege im Ordner *Bruchrechnen* folgende Unterordner an.

- *src*: Hier speichern wir die Quellcode-Dateien für funktionalen Code.
- *test*: Hier werden die Unittests gespeichert.

4.4 Die Pfadangabe für die Python-Module

Damit sich alle Python-Module untereinander kennen, gibt es die Umgebungsvariable `PYTHONPATH`, die wir ergänzen müssen.

- Lege im Hauptordner die Datei `.env` an. Beachte: Der Name beginnt mit einem Punkt.
- Schreibe folgende Zeile in diese Datei: `PYTHONPATH=$PYTHONPATH:../src:../test`

4.5 Die virtuelle Entwicklungsumgebung

Dieser Schritt sollte unbedingt in VS Code erfolgen! Die Einrichtung einer virtuellen Umgebung ohne VS Code kann große Probleme für unser Vorhaben erzeugen.

- Öffne die Befehlspalette.
- Wähle *Python: Umgebung erstellen*.
- Wähle *Venv* aus.
- Wähle den Interpreter `/bin/python3` aus.

Im Hintergrund wird eine virtuelle Umgebung angelegt. Dass es geklappt hat, erkennst du an:

- Im Hauptordner gibt es einen neuen Unterordner `.venv`.
- Wenn du ein neues Terminal startest, dann ist die virtuelle Umgebung aktiviert. Der Pfadname des Terminalprompts beginnt mit `(.venv)`

4.6 Konfiguration der Softwaretest und Installation des Testframeworks

- Öffne die Befehlspalette.
- Wähle *Python: Tests konfigurieren*.
- Wähle *pytest*.
- Wähle *. Root directory*.

Im Hintergrund wird das Testframework Pytest installiert. Prüfe:

- Starte ein Terminal.
- Der Befehl `pip list` sollte `pytest` auflisten.

- Drücke im Navigator auf der linken Seite auf den Erlenmeyerkolben.
 - Zeigt dieser den Hauptordner an? Dann sollte alles in Ordnung sein.
 - Zeigt der Navigator einen Fehler? Dieser sollte nach dem Aktualisieren verschwinden.

4.7 Installation des GUI-Frameworks NiceGui

- Öffne ein Terminal.
- Ist die virtuelle Umgebung aktiviert? Andernfalls bitte aktivieren:
`source .venv/bin/activate`
- Der Befehl `python -m pip install nicegui` installiert das GUI-Framework.

5 Die Softwarearchitektur

5.1 Die Problemstellung

Ein einziges Programm mit vielen Zeilen Code ist unübersichtlich und schwer zu pflegen. Fehler sind schwer zu finden und Änderungen können nur aufgenommen vorgenommen werden.

5.2 Die Lösung

Wir teilen das Programm in verschiedene Einzelmodule auf. Jedes Modul nimmt eine Aufgabe wahr. So wird das Gesamtprogramm viel übersichtlicher und kann besser gewartet werden.

Die Unterteilung in Module und deren Zusammenspiel nennt man **Softwarearchitektur**.

5.3 Trennung von funktionalem Quellcode und Testcode

Eine erste Trennung haben wir bereits bei der Einrichtung der Projektinfrastruktur angelegt:

- Der Ordner *src* wird den Quellcode enthalten, der für das Funktionieren des Programms verantwortlich ist.
- Im Ordner *test* speichern wir die Testprogramme.

5.4 Trennung von Logik und Darstellung

Ein einfaches Architekturprinzip lautet:

Trenne die Programmlogik von der Darstellung auf der Oberfläche.

Wir haben also mindestens zwei Module:

- Ein Modul enthält die Datenstruktur für einen Bruch und die Rechenoperationen mit Brüchen. Das Modul nennen wir *Model*.
- Ein Modul enthält die Oberfläche, mit der die Rechnung dargestellt wird. Dieses Modul nennen wir *View*.

Die Module müssen natürlich zusammen arbeiten, damit das Gesamtprogramm funktioniert. Damit wir trotzdem eine möglichst gute Trennung zwischen den beiden Modulen erhalten, wenden wir ein zweites Architekturprinzip an:

Die View kennt das Model, aber das Model kennt die View nicht.

Wie wir das umsetzen und weshalb das sinnvoll ist, werden wir später sehen.

6 Eine Funktion in Python: Programmierung und Testen

Versuche, die beiden unten angegebenen Aufgaben selbstständig, eventuell im Team oder mit Hilfe des Betreuers zu lösen. Falls du nicht weiter kommst oder dein Ergebnis überprüfen willst, steht im nächsten Kapitel die Musterlösung. (Abschnitt 7)

6.1 Die Funktion zur Berechnung des größten gemeinsamen Teilers

Für das Kürzen von Brüchen benötigen wir die Berechnung des größten gemeinsamen Teilers zweier Ganzzahlen.

Aufgabe:

- Lege in Ordner *src* eine Python-Datei mit dem Namen *mathehelfer.py* an.
- Programmiere darin eine Funktion, die zwei Ganzzahlen als Parameter entgegen nimmt und den zugehörigen ggT berechnet. Namensvorschlag: *berechne_ggt*
- Tipp: Verwende Type-Hints für die Deklaration der Parameter und für den Rückgabebetyp der Funktion.
- Tipp: Zur Berechnung des ggT gibt es den sogenannten Euklidischen Algorithmus in einer iterativen, schnellen Variante.

6.2 Der Unittest

Ein professioneller Softwareentwickler programmiert die Tests für den eigenen funktionalen Quellcode selbst. Das wollen wir uns anschauen. Wir programmieren für jede Funktion eine zugehörige Testfunktion.

Aufgabe:

- Lege im Ordner *test* eine Python-Datei mit dem Namen *test_mathehelfer.py* an.
- Programmiere darin eine Funktion namens *test_berechne_ggt*.

- Überlege: Für welche Werte sollten wir die Berechnung des ggT testen?
- Programmiere die von dir als sinnvoll angesehenen Test jeweils als *assert*-Befehl.
- Öffne die Testansicht (Erlenmeyerkolben im Navigator).
- Führe die Tests aus.
- Sind nur grüne Haken zu sehen? Oder gibt es auch rote Kreuze?

7 Lösung: Programmieren und Testen des ggT

7.1 Die Funktion zur Berechnung des ggT

```
1 def berechne_ggt(a: int, b: int) -> int:
2     while b != 0:
3         (a,b) = (b, a % b)
4     return a
```

python

Die Verwendung eines Tupels in Zeile 3 erspart eine Hilfsvariable zum Vertauschen der Werte.

7.2 Sinnvolle Testwerte

- zwei einfache Werte: $6 = \text{ggT}(12,18)$
- die Werte in umgekehrter Reihenfolge: $6 = \text{ggT}(18,12)$
- zwei größere Zahlen: $13 = \text{ggT}(65,143)$
- ein Faktor ist das Vielfache des anderen Faktors: $8 = \text{ggT}(8,48)$
- eine Primzahl: $1 = \text{ggT}(24,57)$
- zwei Primzahlen: $1 = \text{ggT}(57,86413)$
- der Trivialfall mit der Zahl 1: $1 = \text{ggT}(1,24)$

7.3 Die Testfunktion

```
1 import mathefunktionen
2
3 def test_berechne_ggt():
4     assert 6 == mathefunktionen.berechne_ggt(12,18)
5     assert 6 == mathefunktionen.berechne_ggt(18,12)
6     assert 13 == mathefunktionen.berechne_ggt(65,143)
7     assert 8 == mathefunktionen.berechne_ggt(8,48)
8     assert 1 == mathefunktionen.berechne_ggt(24,59)
9     assert 1 == mathefunktionen.berechne_ggt(59,86413)
10    assert 1 == mathefunktionen.berechne_ggt(1,24)
```

python

8 Bruch: Objektorientierte Analyse und objektorientiertes Design (OOAD)

8.1 Aufgabenstellung OOAD

Diskutiere folgende Fragen im Team:

- Was ist ein Bruch?
- Aus welchen Einzelteilen besteht er?
- Welche Werte sind für Zähler und Nenner erlaubt?
- Dürfen Zähler und Nenner negativ sein?
- Was drückt der Bruchstrich aus?
- Was bedeutet das Kürzen eines Bruchs? Wie geht das?
- Was bedeutet das Erweitern eines Bruchs?
- Wie funktionieren die Grundrechenarten bei Brüchen?
 - Multiplizieren?
 - Dividieren?
 - Addieren?
 - Subtrahieren?

9 Lösung: Objektorientierte Analyse und objektorientiertes Design (OOAD)

9.1 Eigenschaften eines Bruchs

Ein Bruch besitzt folgende Eigenschaften (Attribute):

- Er besteht aus zwei Ganzzahlen, dem Zähler und dem Nenner.
- Der Nenner muss von 0 verschieden sein. Der Versuch, einen Bruch mit dem Nenner 0 zu erzeugen, muss einen Fehler auslösen.
- Zähler oder Nenner dürfen negative Zahlen sein.

9.2 Vorzeichen in Zähler und Nenner

Zähler oder Nenner dürfen negative Zahlen sein. Der Anwender des Programms darf beliebige Zahlen eingeben, solange der Nenner ungleich 0 bleibt.

In der Ergebnisdarstellung möchten wir ein einzelnes negatives Vorzeichen immer in den Zähler schreiben. Tragen sowohl Zähler als auch Nenner ein negatives Vorzeichen, so soll das Ergebnis zwei positive Zahlen besitzen. Das ist eine reine Konvention, damit das Ergebnis ordentlich aussieht.

9.3 Der Bruchstrich

Der Bruchstrich drückt eine Rechenoperation aus, nämlich die Berechnung des Wertes als Kommazahl.

9.4 Die Gleichheit von Brüchen

Mathematisch sind Brüche gleich, wenn sie denselben Wert als Kommazahl ausdrücken:

$$\frac{1}{2} = \frac{2}{4}$$

Dabei ist unerheblich, ob ein Bruch gekürzt oder ungekürzt ist.

In der Programmierung möchten wir strenger sein, damit wir einen Bruch eindeutig identifizieren können. Wir legen dazu fest: In unserem Programmcode gelten zwei Brüche genau dann als gleich, wenn beide Brüche jeweils denselben Zähler und denselben Nenner besitzen.

9.5 Die Unveränderlichkeit von Brüchen

Damit wir die Gleichung

$$\frac{2}{4} = \frac{1}{2}$$

(also die Kürzungsoperation) in unserem Programm ausdrücken können, dürfen wir den Bruch auf der linken Seite im Programmcode nicht verändern. Andernfalls laufen wir Gefahr, dass die Eingabe von

$$\frac{2}{4} =$$

das Ergebnis

$$\frac{1}{2} = \frac{1}{2}$$

zeigt. Das ist zwar richtig, hat aber die linke Seite der Gleichung unbeabsichtigt verändert.

Wir legen also fest:

- Ein Bruch-Objekt ist unveränderlich.
- Jede Rechenoperation mit einem Bruch legt für das Ergebnis einen neuen Bruch an, dessen Zähler und Nenner das Rechenergebnis bilden.

9.6 Das Kürzen von Brüchen

Ein Bruch kann gekürzt vorliegen, d.h. Zähler und Nenner besitzen die minimalen Werte, die dem Kommazahlwert entsprechen.

Das Programm soll und muss auch mit ungekürzten Brüchen arbeiten können. Das erste Zwischenergebnis ist in der Regel ein ungekürzter Bruch

Für das Kürzen eines Bruchs wird zunächst der gemeinsame größte Teiler von Zähler und Nenner bestimmt. Danach werden Zähler und Nenner jeweils durch den ggT geteilt.

9.7 Das Erweitern von Brüchen

Ein Bruch wird erweitert, indem Zähler und Nenner mit demselben Faktor (ungleich 0) multipliziert werden. Das Erweitern wird für die Addition von Brüchen benötigt.

9.8 Das Multiplizieren von Brüchen

Das Multiplizieren ist einfach: Zähler mal Zähler, Nenner mal Nenner.

$$\frac{a}{b} \cdot \frac{c}{d} = \frac{a \cdot c}{b \cdot d}$$

9.9 Das Dividieren von Brüchen

Brüche werden dividiert, indem der erste Faktor mit dem Kehrwert des zweiten Faktors multipliziert wird.

$$\frac{a}{b} \div \frac{c}{d} = \frac{a}{b} \cdot \frac{d}{c}$$

Wir programmieren also eine Funktion zur Bildung des Kehrwertes und führen die Division auf die Multiplikation zurück.

9.10 Das Addieren von Brüchen

Für das Addieren müssen die Nenner gleichnamig gemacht werden, also ein gemeinsamer, der sogenannte Hauptnenner gefunden werden. Ein möglicher Hauptnenner ist das Produkt der beiden Nenner. (Alternativ kann aus den beiden Nenner deren kleinstes gemeinsames Vielfaches (**kgV**) gebildet werden.)

$$\frac{a}{b} + \frac{c}{d} = \frac{a \cdot d}{b \cdot d} + \frac{c \cdot b}{b \cdot d} = \frac{a \cdot d + c \cdot b}{b \cdot d}$$

9.11 Das Subtrahieren von Brüchen

Für das Subtrahieren stellen wir eine ähnliche Überlegung an wie für das Dividieren: Die Subtraktion ist eine Addition, wobei der zweite Summand einem Vorzeichenwechsel vollzieht. Wir programmieren also eine Funktion zum Vorzeichenwechsel bei einem Bruch und führen eine Addition aus.

$$\frac{a}{b} - \frac{c}{d} = \frac{a}{b} + \frac{-c}{d}$$

10 Die Klasse Bruch

Hast du schon einmal objektorientiert mit Klassen und Objekten programmiert?

10.1 Objektorientierung in Python

In Python gibt es zwei unterschiedliche Möglichkeiten, eine Klasse und ihre Attribute zu beschreiben. Wir verwenden hier die Annotation `@dataclass`.

10.2 Die Klassendefinition

Definiere eine Klasse mit Namen *Bruch* mittels der `@dataclass`-Annotation.

Die Objekte der Klasse sollen unveränderlich sein. (*immutable*)

10.3 Die Deklaration der Attribute

Deklariere je ein Attribute für den Zähler und den Nenner.

10.4 Der Konstruktor zur Objekterzeugung

Implementiere die Funktion `__init__()`. Beachte dabei:

- Der Nenner darf nicht 0 sein.
- Wie können die Werte für die Attribute gesetzt werden? Das ist bei unveränderlichen Objekte etwas eigen.

10.5 Eine Funktion zur Berechnung des Zahlenwertes

Implementiere eine Funktion `berechne_wert()`, die für einen Bruch den numerischen Wert berechnet. Was hat diese Funktion mit dem Bruchstrich zu tun?

Teste diese Funktion!

10.6 Eine Funktion für einen positiven Nenner

Grundsätzlich können Zähler oder Nenner negative Zahlen sein. Bei den Ergebnissen möchten wir den Nenner immer positiv haben. Implementiere eine Funktion *normiere_bruch()*, die einen Bruch mit demselben Wert erstellt, dessen Nenner positiv ist.

Teste diese Funktion!

11 Lösung: Die Klasse Bruch

11.1 Die Klassen und ihre Attribute

Die Annotation `@dataclass(frozen=True)` bewirkt zweierlei:

- Die Attribute können explizit mit Namen und Datentyp angegeben werden.
- Die Attribute werden durch *frozen=True* fixiert und können nur einmalig bei der Objekterzeugung gesetzt werden.

```
1 @dataclass(frozen=True)
2 class Bruch():
3
4     zaehler: int
5     nenner: int
```

python

11.2 Der Konstruktor zur Objekterzeugung

Weil die Attribute unveränderliche Werte haben und nur initial in der `__init__()`-Methode gesetzt werden können, ist die Notation in Python etwas umständlich. Falls der Parameter für den Nenner den Wert 0 enthält, wird ein Fehler ausgelöst.

```
1 def __init__(self: Bruch, zaehler: int, nenner: int):
2     if nenner == 0:
3         raise ValueError("Nenner darf nicht 0 sein.")
4     object.__setattr__(self, 'zaehler', zaehler)
5     object.__setattr__(self, 'nenner', nenner)
```

python

11.3 Eine Funktion zur Berechnung des Zahlenwertes

Der Bruchstrich drückt die Division von Zähler durch Nenner aus, also mittelbar den Wert des Bruchs als Kommazahl.

```
1 def berechne_wert(self) -> float:
2     return self.zaehler / self.nenner
```

python

```
1 def test_berechne_wert():
2     assert Bruch(1,2).berechne_wert() == 0.5
3     assert Bruch(1,4).berechne_wert() == 0.25
4     assert Bruch(3,4).berechne_wert() == 0.75
5     assert Bruch(2,1).berechne_wert() == 2.0
6     assert Bruch(5,2).berechne_wert() == 2.5
```

python

11.4 Eine Funktion für einen positiven Nenner

Grundsätzlich können Zähler oder Nenner negative Zahlen sein. Bei den Ergebnissen möchten wir den Nenner immer positiv haben. Implementiere eine Funktion *normiere_bruch()*, die einen Bruch mit demselben Wert erstellt, dessen Nenner positiv ist.

Teste diese Funktion!

12 Die Klasse Bruch: Weitere Funktionen

Programmiere jetzt Funktionen in der Klasse Bruch, um die Grundrechenarten durchführen zu können. Folgende Reihenfolge ist empfehlenswert:

1. Die Funktion zum **Kürzen** eines Bruchs: Beachte, dass der ursprüngliche Bruch unverändert bleibt, der gekürzte Bruch ist ein neues Bruch-Objekt.
2. Die Funktion zum **Multiplizieren** des vorliegenden Bruchs mit einem weiteren Bruch.
3. Eine Funktion zum Bilden des **Kehrwerts** eines Bruchs.
4. Die Funktion zum **Dividieren** des Bruchs durch einen anderen Bruch greift auf die Multiplikation mit dem Kehrwert zurück.
5. Die Funktion zum **Addieren** des Bruchs mit einem anderen Bruch.
6. Eine Funktion, mit der das Vorzeichen des Zählers verändert werden kann.
7. Die Funktion zum **Subtrahieren** greift auf das Addieren zurück, indem beim zweiten Summand das Vorzeichen des Zählers verändert wird.

13 Die grafische Benutzeroberfläche

Jetzt können wir eine grafische Benutzeroberfläche programmieren, mit der die vier Grundrechenarten für Brüche durchgeführt werden können.

Dazu verwenden wir die Grafikbibliothek **NiceGUI**: <https://nicegui.io/>

Schau die Dokumentation von NiceGUI an. Dort wird für alle Bedienelemente anhand eines kleinen Beispiels die Funktionsweise erklärt.

Du benötigst vermutlich folgende Komponenten:

- Number Input: mehrfach für Zähler und Nenner der beiden Brüche und des Ergebnisbruchs
- Dropdown Button zur Auswahl der Rechenoperation $+$, $-$, $\cdot$, $:$
- Button zum Auslösen der Berechnung

Damit die grafischen Komponenten schön ausgerichtet werden, sind vermutlich unterschiedliche **Page Layout**-Elemente hilfreich:

- Column Element
- Row Element
- Grid Element

Fertige zuerst eine Skizze auf Papier an. Überlege, welche Komponenten übereinander und welche Komponenten in einer Reihe stehen sollen.

14 Ausbaustufen

14.1 Eine Klasse für Kommazahlen

Wir möchten mit Kommazahlen ebenso rechnen können wie mit Brüchen. Außerdem sollen

- ein Bruch in eine Kommazahl umgewandelt werden,
- eine Kommazahl in einen Bruch umgewandelt werden,
- Brüche und Kommazahlen miteinander addiert, subtrahiert, multipliziert oder dividiert werden.

Tipp: Der Datentyp **float** in Python bildet die Grundlage für Kommazahlen, lässt sich aber nicht unmittelbar verwenden. Informiere dich über das **Entwurfsmuster Adapter**.

14.2 Eine Klasse für unechte Brüche

Bei einem unechten Bruch ist der Zähler größer als der Nenner. Häufig ist für einen unechten Bruch der Zahlenwert nicht einfach abzulesen. Wie viel ist $\frac{70}{12}$?

Deshalb werden unechte Brüche häufig als **gemischte Zahl** geschrieben:

$$\frac{70}{12} = 5\frac{5}{6}$$

Programmiere eine Klasse für gemischte Zahlen.